PHASE

14

Userland: libc, Shell and Utilities

Build the user-space half of the system: a C library, pipes, a real terminal, the Turk shell, and the everyday commands

Overall progress toward Turk-OS 1.0 (this phase in red) 90 % → 96 %

Duration	3 weeks
Difficulty	★★★★☆ (4/5)
You need	Phase 13: VFS, descriptors, TurkFS; Phase 11: `fork`/`exec`/`wait`, signals
You will have	Turk-libc (system call wrappers with `errno`, strings, `malloc`, buffered `stdio` with `printf`), a `pipe` system call, a terminal line discipline with echo, line editing, Ctrl+C and Ctrl+D, basic ANSI escape codes on the console, `tsh` (the Turk shell) with pipelines, redirection and background jobs, about fifteen core utilities, and a boot that ends at a shell prompt.

1 Why this phase

Until now you have been talking to Turk-OS through a monitor built into the kernel. A real UNIX has no such thing: the shell is an ordinary user program, and so are `ls`, `cat` and everything else you type. That design is only possible because of the system calls you built in Phases 10–13, and this phase is where you find out whether they really work together. OSTEP chapter 5 argues that `fork`, `exec`, `wait` and file descriptors are exactly the right primitives for a shell; now you get to confirm it by writing one.

Most of the code in this phase runs in ring 3. The kernel gains two things: **pipes**, which connect one process's output to another's input, and a proper **terminal** that edits lines and turns Ctrl+C into a signal. Everything else is user space: a C library so programs can be written normally, the shell, and the utilities. When you finish, Turk-OS boots to a prompt, and you can type `ls /bin | wc -l > count.txt`.

2 Concepts you need

2.1 The C library

User programs don't make system calls directly; they call the **C library**, which provides the standard interface (`string.h`, `stdio.h`, `stdlib.h`, `unistd.h`, ...) on top of system calls. Some functions are thin wrappers: `write` puts its arguments in registers and executes `int 0x80`. Others do real work in user space: `printf` formats into a buffer and calls `write` only when the buffer fills or a line ends, and `malloc` manages a heap that it grows with `brk`, reusing your Phase 7 allocator almost unchanged. System calls return a negative error code (`-ENOENT`); by convention the library stores the positive code in the global `errno` and returns -1, which is what programs expect.

2.2 Pipes and redirection

A **pipe** is a kernel buffer with two file descriptors: whatever is written to one end can be read from the other. The shell builds a pipeline with the `fork`/`exec` gap that OSTEP chapter 5 describes: it creates a pipe, forks two children, and in each child uses `dup2` to move the right end of the pipe onto descriptor 0 or 1 before calling `exec` (Figure 1). The programs never know. Redirection (`> file`) is the same trick with a file instead of a pipe. A reader sees end-of-file only when *every* write end is

closed, so every process, the shell included, must close the ends it doesn't use. Forgetting that is the classic pipe bug: the pipeline never finishes.

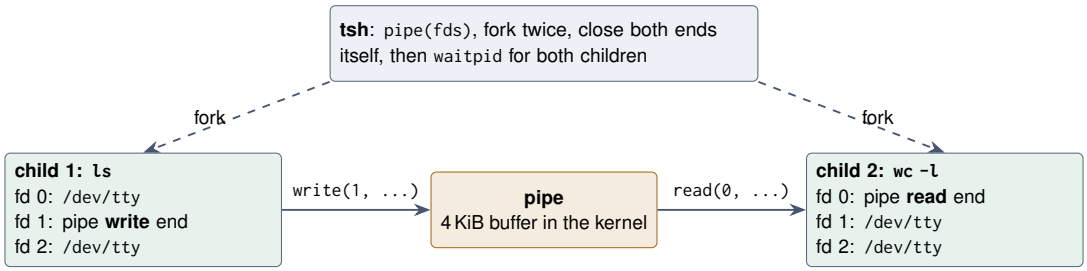


Figure 1: How `ls | wc -l` is wired. Each child `dup2`s its pipe end onto fd 0 or 1 and closes the originals before `exec`.

2.3 The terminal line discipline

When you type at a UNIX prompt, the program doesn't see each key. The kernel's terminal driver collects a whole line, echoes characters, handles Backspace (erase) and Ctrl+U (kill the line), and hands the line over only when you press Enter. This is **canonical** (cooked) mode. Special keys become actions: Ctrl+D at the start of a line means end-of-file, and Ctrl+C sends `SIGINT` to the **foreground** process instead of being passed as input. Programs like editors switch the terminal to **raw** mode to get every key at once. OSDI §3.8.2 describes all of this, and MOS §5.6 covers the output side, including the escape sequences (such as `ESC [2J` to clear the screen) that let programs move the cursor and change colours.

2.4 Every layer at once

This is the phase where a single `printf("merhaba\n")` passes through almost everything you have built (Figure 2). When something prints the wrong thing, walk down this list and check each layer on its own.

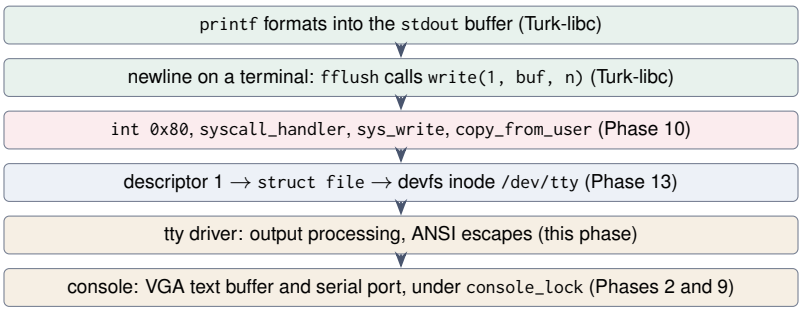


Figure 2: The path of one `printf` in Turk-OS.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	OSTEP	Ch. 5 *Interlude: Process API*; re-read §5.4 on why `fork`/`exec` suits a shell, redirection and pipes	37–48	The design of `tsh` in a few pages.
REQUIRED	LOB	Ch. 11, sections *Using C for User Mode Programs* and *A C Library*	67–68	What a minimal libc needs.
RECOMMENDED	OSDI	§1.3.3 *The Shell*; §1.4 *System Calls*	25–42	The POSIX calls your library wraps, with a sketch of a shell.

Priority	Book	What to read	Pages	Why now
RECOMMENDED	OSDI	§5.6.9 *Pipes and Special Files*	563–565	How MINIX implements pipes and device files.
RECOMMENDED	OSDI	§3.8.2 *Terminal Software*; §3.8.4 *Implementation of the Device-Independent Terminal Driver*	307–316, 331–350	Canonical mode, special characters, and a working line discipline.
RECOMMENDED	MOS	§5.6.1–5.6.2 *Input Software, Output Software*	394–405	Cooked versus raw input, escape sequences.
RECOMMENDED	OSC	§2.1–2.2 *Services, User and OS Interface*; §2.4 *System Services*	55–62, 74–75	Where shells and utilities fit.
OPTIONAL	MOS	§1.5.6 *The Shell*; §10.2 *Overview of Linux* (interfaces, shell, utility programs)	45–46, 723–733	The user's view of a UNIX system.
OPTIONAL	MOS	§12.2 *Interface Design*	985–993	Keep the library and shell simple and orthogonal.
OPTIONAL	LKD	Ch. 5, *APIs, POSIX, and the C Library*	70–71	How a C library sits on top of the system calls.

Real-system lens: how MINIX 3 does it

The MINIX terminal driver, `drivers/tty/tty.c (13600)`, implements canonical mode in `in_process`: it checks every incoming character against the special characters in the terminal's settings (erase, kill, EOF, interrupt, ...), edits the input queue, echoes, and completes a pending `read` when a line is ready. Pipes live in the file-system server, `servers/fs/pipe.c (25900)`: a pipe is an inode whose data blocks act as the buffer, and `suspend/release` put readers and writers to sleep and wake them, just as your wait queues will.

Linux, for comparison. LKD On Linux, the C library wraps every system call and turns a negative return value into -1 plus `errno`, exactly as Turk-libc does.

4 Build it

Task 14.1 Turk-libc: the skeleton

Create `user/libc/` with headers in `user/include/`, built into a static `libc.a` with `i686-elf-ar`. Every program links `crt0.o`, its own objects and `libc.a`. Start with system call wrappers that follow the `errno` convention, then `string.h` and `ctype.h` (copy them from the kernel's `lib/`; you wrote them in Phase 0).

user/libc/unistd.c

```
int errno;

static int ret_errno(int32_t r)            /* kernel returns -E...; libc returns -1 */
{
    if (r < 0) {
        errno = -r;
        return -1;
    }
    return r;
}

int read(int fd, void *buf, size_t n)  { return ret_errno(syscall3(SYS_read,  fd, (uint32_t)buf, n)); }
int write(int fd, const void *buf, size_t n) { return ret_errno(syscall3(SYS_write, fd, (uint32_t)buf, n)); }
    ↪
```

```
int open(const char *path, int flags)  { return ret_errno(syscall3(SYS_open, (uint32_t)path, flags, 0)); }
int pipe(int fds[2])                   { return ret_errno(syscall3(SYS_pipe, (uint32_t)fds, 0, 0)); }
```

Task 14.2 malloc in user space

Port your Phase 7 heap: replace `ksbrk` with the `brk` system call and `ASSERT` with an `abort()` that prints a message and exits with `SIGABRT`. Add `calloc`, `realloc` and `free`. Because `brk` allocates lazily (Phase 11), a program that asks for a large heap but touches little of it stays cheap.

Verify: The heap stress test from Phase 7, compiled as a user program, passes.

Task 14.3 stdio

Implement `FILE` with a buffer, a mode (unbuffered, line-buffered, fully buffered) and a descriptor, plus `stdin`, `stdout` and `stderr`. `stdout` is line-buffered when it is a terminal (check with an `isatty` built on `fstat`) and fully buffered otherwise; `stderr` is unbuffered. Port your `kvprintf` engine for `printf`, `fprintf`, `sprintf` and `snprintf`, and add `fopen`, `fclose`, `fgets`, `fputs`, `fread`, `fwrite`, `fflush`, `getchar` and `putchar`. Make `exit` flush all open `FILE`s before calling the system call, while `_exit` doesn't.

Verify: `printf` output appears without an explicit `fflush` when printing to the terminal, and is complete in a file even when the program ends with `return 0` from `main`.

Task 14.4 Pipes in the kernel

Add `sys_pipe`: create a pipe object with a 4 KiB ring buffer, counts of open readers and writers, a spinlock and two wait queues, then two `struct file`s (read end and write end) and two descriptors. A read from an empty pipe sleeps until data arrives, or returns 0 (end-of-file) if there are no writers left. A write to a full pipe sleeps until there is room; if no readers are left, it sends `SIGPIPE` to the writer and returns `-EPIPE`. Closing an end decrements its count and wakes the other side.

kernel/fs/pipe.c

```
#define PIPE_SIZE 4096

struct pipe {
    uint8_t           buf[PIPE_SIZE];
    uint32_t          rpos, wpos;          /* bytes in pipe = wpos - rpos */
    int               readers, writers;    /* open ends                   */
    spinlock_t        lock;
    struct wait_queue can_read, can_write;
};

int32_t pipe_read(struct pipe *p, uint8_t *dst, uint32_t n)
{
    spin_lock(&p->lock);
    while (p->wpos == p->rpos) {           /* empty */
        if (p->writers == 0) {             /* nobody can ever write again */
            spin_unlock(&p->lock);
            return 0;                      /* end-of-file */
        }
        sleep_on(&p->can_read, &p->lock);
    }
    uint32_t got = 0;
    while (got < n && p->rpos != p->wpos)
        dst[got++] = p->buf[p->rpos++ % PIPE_SIZE];
    wake_up_all(&p->can_write);
    spin_unlock(&p->lock);
    return got;
}
```

The loop copies into a kernel buffer; `sys_read` then hands it to the user with `copy_to_user` after the lock is released, because uaccess may fault and may allocate a frame (Phase 11), so it is never called with a spinlock held. That kernel buffer has a fixed size, so `sys_read` and `sys_write` on a pipe first clamp the count to `PIPE_SIZE`; a larger request returns a short count, as UNIX allows, and never overruns the buffer.

Verify: A test program writes 1 MiB into a pipe from a child and reads it back in the parent, checking every byte; the writer blocks whenever the buffer is full. A single `read` of 1 MiB returns at most 4096 bytes.

Task 14.5 A real terminal

Turn `/dev/tty` into a proper terminal driver. Keyboard characters go into an input queue. In canonical mode, echo them; Backspace removes the last character from the current line and from the screen; Ctrl+U removes the whole line; Enter completes the line and wakes readers; Ctrl+D on an empty line makes the pending `read` return 0; Ctrl+C sends `SIGINT` to the foreground process group and discards the line. Add an `ioctl` (number 54) so a program can read and set a flags word (canonical on or off, echo on or off) and set the foreground process group, which the shell does before running a job and after it finishes. On the output side, teach the console the few ANSI sequences programs need: `ESC [2J` (clear), `ESC [H` and `ESC [row;colH` (move the cursor), `ESC [K` (clear to end of line) and `ESC [...m` (colours). The input queue holds one line of `MAX_CANON` (256) bytes: further characters are dropped with a beep, and `read` and `write` on the terminal clamp the count to the size of the kernel buffer they copy through, exactly as the pipe does.

Verify: A program that reads lines and prints them in capitals works like a normal UNIX filter: editing works, Ctrl+D ends it, Ctrl+C interrupts it.

Task 14.6 tsh, the Turk shell

Write `user/sh/tsh.c`. Read a line, split it into words (honouring single and double quotes), and parse it into one or more pipelines separated by `;`. Each pipeline is a list of commands, each with its own redirections (<, >, >>), plus an optional `&` for background. Built-ins run inside the shell because they must change its own state: `cd`, `pwd`, `exit`, `export` and `help`. Everything else runs with `fork` and `exec`, looked up in the directories of `PATH` (default `/bin`). Before each prompt, reap finished background jobs with `waitpid(-1, &status, WNOHANG)`. The shell ignores `SIGINT` itself and makes each foreground job the terminal's foreground group, so Ctrl+C reaches only the job. The core of the pipeline code looks like this:

user/sh/tsh.c (running a pipeline)

```
static void run_pipeline(struct cmd *cmds, int n, bool background)
{
    int   prev_read = -1;
    pid_t pids[MAX_CMDS];

    for (int i = 0; i < n; i++) {
        int fds[2] = { -1, -1 };
        if (i < n - 1 && pipe(fds) < 0) { perror("pipe"); return; }

        pid_t pid = fork();
        if (pid == 0) {                                  /* child */
            if (prev_read != -1) { dup2(prev_read, 0); close(prev_read); }
            if (i < n - 1)       { dup2(fds[1], 1); close(fds[0]); close(fds[1]); }
            apply_redirections(&cmds[i]);                /* <, >, >> override the pipe */
            execvp(cmds[i].argv[0], cmds[i].argv);
            perror(cmds[i].argv[0]);
            _exit(127);                                  /* exec failed */
        }
```

```
        pids[i] = pid;
        if (prev_read != -1) close(prev_read);           /* the shell keeps no pipe ends */
        if (i < n - 1) { close(fds[1]); prev_read = fds[0]; }
    }
    if (!background)
        for (int i = 0; i < n; i++)
            waitpid(pids[i], NULL, 0);
}
```

Verify: `echo merhaba | wc -c` prints 8; `ls /bin > /mnt/list.txt` followed by `cat < /mnt/list.txt` shows the listing; `sleep 5 &` returns to the prompt at once and is reaped later.

Task 14.7 Core utilities

Write each as a small program in `user/bin/`. Keep them simple and predictable; they double as tests for the kernel.

Group	Programs and what they exercise
text	`echo`, `cat` (files or stdin), `wc` (`-l -w -c`), `hexdump`: read, write, pipes
files	`ls` (with `-l`), `mkdir`, `rm`, `cp`, `mv`, `touch`: `getdents`, `stat`, `open` flags, `unlink`, `rename`
processes	`ps`, `kill`, `sleep`: a `getprocs` system call that copies a table of task information to the user, signals, timers
system	`uptime`, `clear` (ANSI escape), `sync`, `shutdown` (sync, then power off QEMU with `outw(0x604, 0x2000)` in the kernel)

Task 14.8 Boot into the shell

Make `init` open `/dev/tty` for descriptors 0–2, print a banner, start `tsh`, and start a new one whenever it exits, while still reaping orphans. Keep the old kernel monitor reachable for emergencies, for example on the serial port or with a `monitor` boot option, but only in debug and test builds: compile its files only when the Makefile defines `CONFIG_MONITOR`, which the normal and test builds do and the release build of Phase 15 does not. A command interpreter inside the kernel, with `crash` and `reboot`, has no place on a system that other people use. Code page 437 has no ş, so a banner like “Turk-OS 1.0 – Hos geldiniz!” stays ASCII until Phase 15.

Verify: Turk-OS boots straight to `turk@os:/$`, and typing `exit` brings back a fresh shell. A kernel built without `CONFIG_MONITOR` has no monitor symbols: `i686-elf-nm build/kernel.elf | grep monitor` prints nothing.

Task 14.9 Test the userland

Add a `-c` option to `tsh` (run one command line and exit) and a test runner that executes a list of command lines and compares their output with expected text. Put these cases in `make test`.

Command line	Expected output
`echo a b c`	`a b c`
`echo merhaba | wc -c`	`8`
`echo x > /mnt/t; cat /mnt/t; rm /mnt/t`	`x`
`cat /etc/motd | cat | cat | wc -l`	the number of lines in `motd`
`nosuchprogram`	an error message, exit status 127

Task 14.10 Limits, and the fuzzer again

Every resource a process can ask for needs a fixed upper bound, or one process can take all of it. Collect the limits in `include/limits.h`, which the kernel and Turk-libc share, check each one where the resource is taken, and copy the table into your notes.

Limit	Value	When it is exceeded
`OPEN_MAX`, open files per process	64	`open`, `dup` and `pipe` return `-EMFILE`
`CHILD_MAX`, processes per user	64	`fork` returns `-EAGAIN`; one user until Phase 15
`PIPE_SIZE`, pipe buffer	4096	`read` and `write` return a short count
`MAX_CANON`, terminal line	256	extra characters are dropped
`PATH_MAX`, path length	256	`-ENAMETOOLONG`
`ARG_MAX`, `argv` and `envp` for `execve`	32 KiB	`-E2BIG`

Then extend `user/bin/fuzz` once more: create pipes and close random ends, read and write pipes with huge counts, call `ioctl` 54 on the terminal and on other descriptors with random requests and arguments, and `dup2` onto random descriptors. Run it in the nightly CI job together with the file calls from Phase 13.

Verify: Once a process holds 64 descriptors, counting 0–2, the next `open` returns `-EMFILE`; `fork` beyond `CHILD_MAX` returns `-EAGAIN`; a path of 300 characters gives `-ENAMETOOLONG`; `fuzz` runs for 10 minutes without a kernel panic.

5 Testing and debugging

Symptom	Likely cause	What to do
A pipeline never finishes	Some process (often the shell) still holds a pipe write end, so the reader never sees end-of-file	Close every unused end in every process; list open descriptors with a debug command.
Output from `printf` is missing	Buffered data never flushed, because the program called `_exit` or crashed	`exit` must flush; use `stderr` for debug output.
Ctrl+C kills the shell	The shell didn't ignore `SIGINT`, or it stayed the foreground group	Ignore `SIGINT` in `tsh`; set the job as foreground before `exec` and take it back after `wait`.
`ls` in a pipe prints nothing, but works alone	The program writes to the terminal directly instead of to descriptor 1	All output goes through `write(1, ...)`; only the tty driver knows about the screen.
Zombies accumulate after background jobs	Background children are never waited for	Reap with `WNOHANG` before every prompt.
Arguments arrive garbled in `main`	Wrong `argv` layout on the initial stack or missing NULL terminator	Compare the stack with the Phase 11 diagram in GDB.

6 Milestone

Milestone: A usable system

- ☐ Turk-libc provides system call wrappers with `errno`, strings, `malloc` and buffered `stdio`.
- ☐ Pipes, the terminal line discipline and ANSI output work.
- ☐ Turk-OS boots to a `tsh` prompt; pipelines, redirection, background jobs and Ctrl+C work.

- ☐ The core utilities are present and pass the scripted tests.
- ☐ No zombies remain after a long interactive session.
- ☐ Every limit in the limits table is enforced; pipe and terminal reads and writes clamp the count to the kernel buffer size.
- ☐ `fuzz` covers pipes and terminal calls; the kernel monitor is compiled only into debug and test builds.

Self-check questions

1. Why does the shell `fork` before `exec`? What would break if it called `exec` directly?
2. Explain exactly how `ls | wc` is wired with `pipe`, `fork` and `dup2`.
3. Why must unused pipe ends be closed, and by whom?
4. Why is `cd` a built-in and not a program in `/bin`?
5. Canonical versus raw mode: who implements line editing in Turk-OS, and why not the shell?
6. List every layer a `printf` passes through on its way to the screen.

7 Going further

Add job control: process groups, `fg`, `bg` and `jobs`, and Ctrl+Z sending `SIGTSTP`. Give `tsh` variables, `if`, `for` and scripts run from files. Write a small full-screen text editor that uses raw mode and ANSI escapes. Or port an existing program with modest needs; a small interpreter such as Lua is a classic target, and the missing libc functions it asks for will show you what your library still lacks.

Next: **Phase 15 Hardening, Polish and Release: Turk-OS 1.0** →

Books used in this plan

LOB — Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP — Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC — A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS — Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI — Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD — Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP — Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. LOB and OSTEP are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.